

Securing PLC-to-AI Data Pipelines in Mission-Critical Industrial Environments Using PKI

Parastoo Shojaei Sarcheshmeh

University of Northampton

Student ID: 25828319

parastoo.shojaeisarcheshm25@my.northampton.ac.uk

Abstract— Industrial control systems rely on data pipelines that connect Programmable Logic Controllers (PLCs) to Artificial Intelligence (AI) analytics platforms for real-time monitoring and decision-making. A PLC is a specialized industrial computer that is used to control machines and physical processes like temperature control, pressure monitoring, and alarm management in places like power plants, water treatment plants, and manufacturing lines. Even though the data they handle is very important, these pipelines are often not encrypted, which means that network attackers can easily access and change sensitive operational data.

This paper presents a practical security analysis of a PLC-to-AI data pipeline built using OpenPLC Runtime v4 and a Python Flask server. Two scenarios are implemented and compared: an insecure HTTP pipeline, in which all sensor data including temperature readings, alarm states, and system status is captured in plaintext using Wireshark; and a secure HTTPS pipeline protected by TLS 1.3 and an X.509 digital certificate using RSA 4096-bit encryption. Public Key Infrastructure (PKI) is used to establish trust, authenticate the server, and encrypt all data in transit.

Results confirm that TLS 1.3 simultaneously provides confidentiality, integrity, and authentication, rendering all intercepted traffic completely unreadable without the private key. Findings are mapped to ISO 27001/27002 security controls, and practical industry recommendations are provided covering certificate management, mutual TLS, and network monitoring.

Keywords— Programmable Logic Controllers, PKI, TLS 1.3, X.509, PLC security, industrial cybersecurity, OT/IT integration, network encryption, Wireshark, ISO 27001, RSA

I. INTRODUCTION

Industries like factories, power plants, water treatment facilities, and oil refineries are changing fast. They now use data pipelines to connect physical control systems to AI platforms in the cloud. These pipelines carry live sensor readings temperature, pressure, flow rates, alarm states that help automated systems make decisions about critical infrastructure. This makes pipeline security not just a technical issue, but a matter of public safety.

At the core of most industrial control systems is the Programmable Logic Controller (PLC).

A PLC is a rugged industrial computer that reads data from physical sensors, runs a control program, and sends commands to devices like motors, valves, and heaters. PLCs run in real time and are built to handle tough conditions. They sit at the lowest level of what we call Operational Technology

(OT) the systems that monitor and control physical processes as opposed to Information Technology, which handles data and communication.

As industries become more connected, OT and IT networks are increasingly being integrated. PLC data is now sent over standard IP networks to AI systems that detect faults, predict maintenance needs, and automate control decisions. However, this brings serious security risks. Many industrial protocols were built decades ago with no security in mind they transmit data as plain text with no encryption, authentication, or integrity checks.

The impact of this can be serious. In 2021, a water treatment plant in Oldsmar, Florida was accessed remotely by an attacker who used the plant's own systems to push sodium hydroxide levels to over 100 times the safe limit. A worker caught it in time, but the incident showed clearly that unprotected industrial pipelines can put lives at risk. Similar cases have been reported across power grids, factories, and transport networks worldwide.

The answer is to apply the same security principles used to protect internet traffic to industrial pipelines. Public Key Infrastructure (PKI) uses digital certificates and asymmetric cryptography to provide three key guarantees: confidentiality (data cannot be read by outsiders), integrity (data cannot be changed without detection), and authentication (both sides can verify who they are talking to).

This paper tests whether PKI and TLS can be practically applied to a PLC-to-AI pipeline in a lab setting. Using OpenPLC Runtime v4 as a PLC simulator and Python Flask as an AI server, we build two scenarios: one insecure pipeline using plain HTTP, and one secure pipeline using HTTPS with TLS 1.3 and an X.509 certificate with RSA 4096-bit encryption. Wireshark is used to show exactly how much data is exposed in each case.

II. LITERATURE REVIEW

PLCs have been widely studied from a security perspective. Alsabbagh et al. [1] looked specifically at OpenPLC — the same platform used in this project — and found several serious issues including weak communication protocols, lack of encryption, and risks to data integrity. This paper builds directly on that work by showing how TLS encryption can fix the data-in-transit problem they highlighted.

Ma et al. [2] studied twenty-three commercial PLCs from thirteen vendors over three years and found that even modern PLCs still lack basic confidentiality and integrity protections.

This confirms why applying encryption at the application layer, as done in this paper, is necessary.

The scale of the problem is well documented. Barbieri et al. [3] analysed real ICS traffic and found that 75.6% of ICS systems communicate without any encryption or integrity protection. Nawrocki et al. [4] found similar results, noting that industrial protocols were never built for open networks a design choice that has become a serious liability as these systems connect to the internet.

Dahlmanns et al. [5] found that many industrial IoT devices actually support TLS but simply leave it turned off. Their work is a key motivation for this paper, which shows that switching on TLS 1.3 in an existing pipeline requires very little effort.

Other studies confirm TLS works well in industrial settings. Liu et al. [7] deployed a TLS-based authentication system in a real SCADA network and showed it successfully blocked man-in-the-middle attacks. AlSaid et al. [8] explored adding encryption directly into PLC applications at design time, though their focus was on PLC-to-PLC communication rather than PLC-to-AI server communication as studied here.

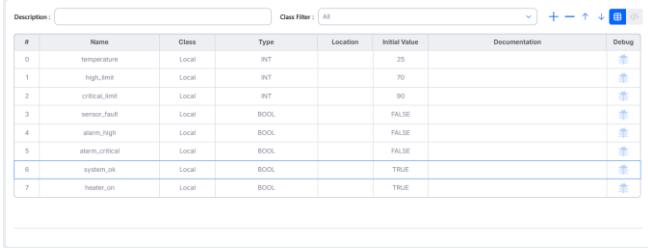
Three things are clear from the literature: encryption is missing from most industrial systems, TLS is an effective fix, and OpenPLC is a well-established research tool. What is missing is a study that combines all three using OpenPLC with a Flask AI endpoint, showing Wireshark captures for both HTTP and HTTPS, and linking the results to ISO 27001/27002. This paper fills that gap.

III. TECHNICAL BACKGROUND

A Programmable Logic Controller (PLC) is a special type of industrial computer that is designed to control machines and physical processes in real time. Unlike a regular computer, a PLC is built to run non-stop in tough industrial environments it can handle extreme heat, electrical interference, and vibration without failing. It works by reading data from sensors (such as temperature or pressure sensors), running a control program, and then sending commands to physical devices like motors, valves, or heaters.

PLCs are programmed using standard languages defined by IEC 61131-3. The most common ones are Ladder Diagram (LD) and Structured Text (ST), which is similar to Pascal or C. PLCs are used in many critical industries power plants use them to control turbines, water treatment plants use them to manage chemical levels, factories use them on production lines, and oil and gas facilities use them to monitor pipelines.

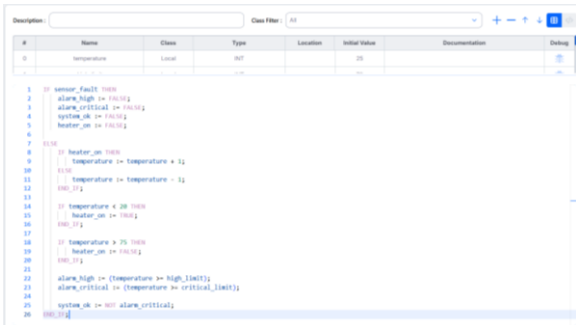
In this project, we use OpenPLC Runtime v4 as our PLC. OpenPLC is a free, open-source PLC that runs on a regular computer, which means we do not need expensive physical hardware. OpenPLC provides a REST API on port 8443 over HTTPS, which allows a Python script to read the PLC's variable values in real time. Our PLC program simulates a temperature monitoring system with a heater and alarm logic. It uses eight variables: temperature (INT, starting at 25°C), high_limit (INT, 70°C), critical_limit (INT, 90°C), and five boolean flags — sensor_fault, alarm_high, alarm_critical, system_ok, and heater_on.



#	Name	Class	Type	Location	Initial Value	Documentation	Default
0	temperature	Local	INT		25		
1	high_limit	Local	INT		70		
2	critical_limit	Local	INT		90		
3	sensor_fault	Local	BOOL		FALSE		
4	alarm_high	Local	BOOL		FALSE		
5	alarm_critical	Local	BOOL		FALSE		
6	system_ok	Local	BOOL		TRUE		
7	heater_on	Local	BOOL		TRUE		

Fig 1. OpenPLC variable table

The control logic is written in Structured Text. If a sensor fault is detected, all alarms are turned off and the heater is disabled. Otherwise, the heater turns on when temperature drops below 20°C and turns off above 75°C. The alarm_high flag is set when temperature goes above the high_limit, and alarm_critical is set above the critical_limit. This gives us realistic sensor data to send through our pipeline.



```

1 sensor_fault := TRUE;
2 alarm_high := FALSE;
3 alarm_critical := FALSE;
4 system_ok := TRUE;
5 heater_on := TRUE;
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26

```

Fig 2. OpenPLC program

In industrial settings, there are two types of technology networks. Operational Technology (OT) covers the systems that control physical processes like PLCs and SCADA systems. Information Technology (IT) covers the systems that handle data like servers, databases, and AI platforms. In the past, OT systems were kept completely separate from IT networks, which gave them a basic level of protection. But today, companies want to connect PLC data directly to AI and cloud platforms for real-time monitoring and predictive maintenance. This connection between OT and IT creates serious security risks, because most industrial protocols like Modbus TCP were never designed with security in mind they send data as plain text with no encryption or authentication.

Cryptography is the science of protecting information by converting it into a form that only authorised parties can read. There are two main types used in modern security. Symmetric encryption uses one shared secret key to both encrypt and decrypt data AES (Advanced Encryption Standard) is the most common example, and it is very fast, making it great for encrypting large amounts of data. Asymmetric encryption uses two mathematically linked keys: a public key that anyone can see, and a private key that only the owner keeps. Data encrypted with the public key can only be decrypted with the private key. RSA is the most well-known asymmetric algorithm in this project we use a 4096-bit RSA key, which is considered extremely secure. Hash functions like SHA-256 are also important they produce a fixed-length fingerprint of any data, and if even one byte is changed, the fingerprint changes completely, which helps detect tampering.

Public Key Infrastructure (PKI) is the system that manages digital certificates and makes asymmetric encryption practical. At the centre of PKI is the Certificate Authority a trusted organisation that issues digital certificates. A certificate proves that a public key really belongs to a specific server or organisation. When a client connects to a server, the server sends its certificate. The client checks that the certificate was signed by a trusted CA, that it has not expired, and that it has not been revoked. If everything checks out, the client knows it is talking to the real server and not an attacker. In this project we use a self-signed certificate, which means the server signs its own certificate instead of using a CA. This is common in research environments and provides the same level of encryption, but in a real production system a proper CA-signed certificate should be used.

An X.509 certificate is the standard format for digital certificates used in PKI. It contains several important fields: the Subject (who the certificate belongs to — in our case CN=localhost, O=PLC Security Project), the Issuer (who signed it — same as subject for self-signed), the Public Key (our RSA 4096-bit key), the Validity Period (365 days, from April 2026 to April 2027), the Subject Alternative Name or SAN (which allows the certificate to work for both DNS:localhost and IP:127.0.0.1), and a Digital Signature that proves the certificate has not been changed. We generated our certificate using OpenSSL with this command:

```
openssl req -x509 -newkey rsa:4096 -sha256 -nodes
-keyout key.pem -out cert.pem -days 365
-subj "/CN=localhost/O=PLC Security Project/C=GB"
```

This gives us two files: `cert.pem` (the certificate that is sent to clients) and `key.pem` (the private key that stays on the server and is never shared).

[illegible]

Fig 3. X.509 certificate details

Transport Layer Security (TLS) is the protocol that encrypts most internet traffic today it is what makes HTTPS work. TLS sits between the TCP network layer and the application layer, creating a secure tunnel for data to travel through. TLS 1.3 is the latest version, released in 2018 as RFC 8446. It is faster and more secure than the previous version (TLS 1.2) because it completes the connection setup in just one round trip instead of two, and it removes older, weaker encryption algorithms. Here is how the TLS 1.3 handshake works: first, the client sends a Client Hello message with the encryption methods it supports and a temporary public key; the server replies with its own temporary key and its X.509 certificate; both sides then independently calculate the same session key without ever sending it over the network; the client checks the certificate and confirms the handshake is complete; and from this point on, all data is encrypted using AES-256-GCM. TLS also supports Perfect Forward Secrecy (PFS), which means that even if the server's private key is stolen in the future, past communications cannot be decrypted because each session uses a different temporary key. TLS

provides three core security guarantees: Confidentiality (data is encrypted and cannot be read by anyone on the network), Integrity (any changes to the data in transit are detected), and Authentication (the certificate proves the server is who it claims to be).

IV. SYSTEM ARCHITECTURE

This project implements two versions of a PLC to AI data pipeline – one insecure and one secure to demonstrate the difference that PKI and TLS make in a real industrial communication scenario. Both scenarios share the same three-component architecture: an OpenPLC Runtime acting as the industrial PLC, a Python bridge script that reads data from the PLC and forwards it, and a Flask server that acts as the AI analytics endpoint.

In the real world, this kind of pipeline would connect a physical industrial PLC (such as a Siemens S7 or Allen-Bradley ControlLogix) to a cloud-based AI platform over a secured network using VPNs, firewalls, and encrypted protocols like OPC-UA. In our implementation, we replicate the same architecture and security concepts using software tools on a single machine, which allows us to demonstrate real network vulnerabilities and their mitigations without needing expensive industrial hardware.

The three components work as follows. First, OpenPLC Runtime v4 runs the PLC program and exposes a REST API on port 8443 over HTTPS. Second, the Python bridge script (`plc_bridge.py` or `plc_bridge_secure.py`) authenticates with OpenPLC using a JWT token, reads the PLC status and sensor values every two seconds, and forwards the data as a JSON payload to the Flask server. Third, the Flask server (`flask_server.py` or `flask_server_secure.py`) receives the data and stores it representing the AI analytics endpoint that would process the data in a real deployment.

The key difference between the two scenarios is the protocol used between the bridge script and the Flask server. In Scenario 1 (insecure), the bridge sends data over plain HTTP to port 5000 with no encryption, no authentication, fully visible on the network. In Scenario 2 (secure), the bridge sends data over HTTPS to port 5443, protected by TLS 1.3 and verified using the X.509 certificate generated by `generate_cert.py`.

The data payload transmitted in both scenarios is identical in structure it contains the cycle number, PLC status, temperature reading, heater state, alarm flags, and overall system health. This is exactly the kind of sensitive operational data that must be protected in a real industrial environment. The difference between the two scenarios is not what data is sent, but how it is sent in the insecure scenario this payload is fully readable by any attacker on the network, while in the secure scenario it is completely encrypted and unreadable without the private key.

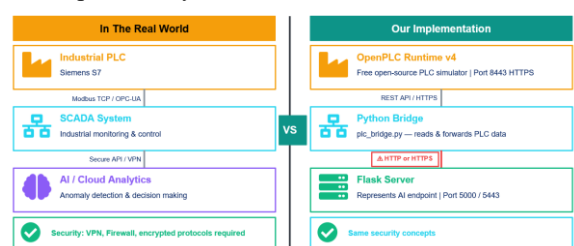


Fig 4. System architecture comparison: real-world industrial pipeline versus our research implementation

V. IMPLEMENTATION

This section describes how both scenarios were built and how each component works. The implementation is divided into two parts: Scenario 1 (insecure HTTP) and Scenario 2 (secure HTTPS/TLS).

A. Scenario 1 — Insecure HTTP Pipeline

The insecure scenario consists of two Python scripts: `flask_server.py` and `plc_bridge.py`.

`flask_server.py` is a simple Flask web server that listens on port 5000 over plain HTTP. It has one main endpoint `POST /plc-data` — which receives JSON data from the bridge script, stores it in memory, and prints a summary to the console. There is no encryption, no certificate, and no authentication on this server. Any machine on the same network can send data to it or intercept the traffic passing through it.

```
if __name__ == "__main__":
    print("=" * 55)
    print(" Flask PLC Data Receiver - INSECURE (HTTP) MODE")
    print("=" * 55)
    print(" Protocol : HTTP (no encryption)")
    print(" Port : 5000")
    print(" WARNING : All data is transmitted as plain text!")
    print(" Endpoint : http://127.0.0.1:5000/plc-data")
    print("=" * 55)

    # Run on all network interfaces so it can receive from the bridge script.
    # debug=False is important for any demo/production-like environment.
    app.run(host="0.0.0.0", port=5000, debug=False)
```

Fig5. `flask_server.py` startup configuration showing HTTP mode on port 5000 with no SSL context.

`plc_bridge.py` is the script that connects the two sides of the pipeline. It first authenticates with the OpenPLC Runtime by sending a POST request to the `/api/login` endpoint and receiving a JWT access token. This connection to OpenPLC uses HTTPS because OpenPLC already has its own certificate on port 8443 so this part is secure. The bridge then enters a loop that runs every two seconds: it reads the PLC status, builds a JSON payload with the sensor values, and sends it to the Flask server over plain HTTP. This is the vulnerable part the data leaves the bridge completely unencrypted and can be captured by anyone running Wireshark on the network.

```
# Flask data receiver server (uses plain HTTP - INSECURE)
FLASK_URL = "http://127.0.0.1:5000"

response = requests.post(
    f"{FLASK_URL}/plc-data",
    json=data,
)
```

Fig 6. `plc_bridge.py` showing `FLASK_URL` set to plain HTTP and POST request with no encryption.

B. Scenario 2 — Secure HTTPS/TLS Pipeline

The secure scenario adds PKI and TLS to the same pipeline using three files: `generate_cert.py`, `flask_server_secure.py`, and `plc_bridge_secure.py`.

The first step is to run `generate_cert.py`, which uses OpenSSL to create a self-signed X.509 certificate and RSA 4096-bit private key. This script runs the OpenSSL command programmatically and produces two files: `cert.pem` (the public certificate that is sent to clients) and `key.pem` (the private key that stays on the server and is never shared). These two files are the PKI foundation of the entire secure scenario.

```
cmd = [
    "openssl",
    "req",
    "-x509",
    "-newkey",
    "rsa-4096",
    "-sha256",
    "-nodes",
    "-keyout",
    KEY_FILE,
    "-out",
    CERT_FILE,
    "-days",
    "365",
    "-subj",
    "/CN=localhost/O=PLC Security Project/C=GB",
    "-addext",
    "subjectAltName=DNS:localhost,IP:127.0.0.1",
]
```

Fig 7. `generate_cert.py` OpenSSL command generating certificate

`flask_server_secure.py` is almost identical to the insecure version, with one critical difference — it passes the certificate and key to Flask's `ssl_context` parameter. This single line is all it takes to upgrade the server from plain HTTP to HTTPS with TLS 1.3. Flask handles the entire TLS handshake automatically.

```
# Run with TLS using the generated certificate and private key
app.run(
    host="0.0.0.0",
    port=5443,
    ssl_context=(CERT_FILE, KEY_FILE), # Enable TLS
    debug=False,
)
```

Fig 8. `flask_server_secure.py` running on HTTPS port 5443

`plc_bridge_secure.py` connects to the secure Flask server using HTTPS instead of HTTP. It passes the `cert.pem` file to the `verify` parameter, which tells the requests library to verify the server's certificate before sending any data — this is the authentication step that prevents man-in-the-middle attacks.

The bridge also sets the `SSLKEYLOGFILE` environment variable before making any requests, which tells Python to log the TLS session keys to a file. This allows Wireshark to decrypt the captured traffic for verification purposes proving that the data is actually encrypted and can only be read with the key.

```
# OpenPLC Runtime API (HTTPS)
PLC_URL = "https://127.0.0.1:8443"

# Flask secure server (HTTPS - encrypted)
FLASK_URL = "https://127.0.0.1:5443"

# Path to the self-signed certificate for verification
CERT_FILE = "cert.pem"
```

```
response = requests.post(
    f"{FLASK_URL}/plc-data",
    json=data,
    verify=CERT_FILE, # Verify server identity using our certificate
)
```

Fig 9. `plc_bridge_secure.py` connecting to Flask over HTTPS with certificate verification enabled

VI. RESULTS AND ANALYSIS

This section presents the results of both scenarios and analyses what the network traffic captures show. The results are based on live Wireshark captures taken during the execution of both pipelines.

A. Scenario 1 — Insecure HTTP: Full Data Exposure

When `plc_bridge.py` sends data to `flask_server.py` over HTTP, the entire JSON payload is visible on the network as plain text. Wireshark captures every packet and displays the full contents with zero effort no decryption. Any attacker on the

same network segment can see exactly what the PLC is reporting in real time.

As shown in Fig. 10, the Wireshark capture on port 5000 reveals the following information clearly: the protocol is HTTP/1.1, the endpoint is POST /plc-data, and the full JSON payload is readable in both the packet list and the hex panel. The attacker can see the exact temperature reading, whether the heater is on or off, whether any alarms have triggered, and the overall system status. This is a complete and total exposure of the industrial data pipeline.

This result demonstrates three specific security failures. First, there is no confidentiality the data is readable by anyone on the network. Second, there is no integrity protection an attacker could modify the JSON values in transit and the server would have no way to detect the change. Third, there is no authentication there is nothing stopping an attacker from sending fake data to the Flask server pretending to be the bridge, or setting up a fake Flask server to intercept the bridge's data.

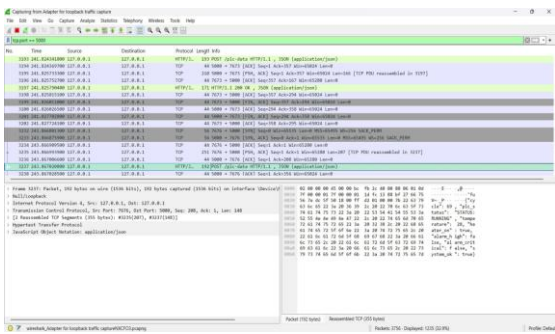


Fig 10. Wireshark capture of HTTP traffic on port 5000, showing PLC sensor data fully visible as plain

B. Scenario 2 — Secure HTTPS/TLS: Complete Protection

When `plc_bridge_secure.py` sends data to `flask_server_secure.py` over HTTPS, the result is completely different. Wireshark can still capture the packets, but there is nothing useful to see. The protocol shown is TLSv1.3, not HTTP. There is no POST /plc-data visible, no JSON payload, no temperature values, no alarm states. The hex panel shows only random binary data that is completely unreadable without the private key.

As shown in Fig. 11, the Wireshark capture on port 5443 shows only TLSv1.3 Application Data packets. The TLS handshake is visible at the start of the connection this is normal and expected, as the handshake itself does not contain any application data. After the handshake completes, all subsequent packets contain only encrypted binary content. An attacker capturing this traffic gains absolutely no useful information about the PLC's operational state.

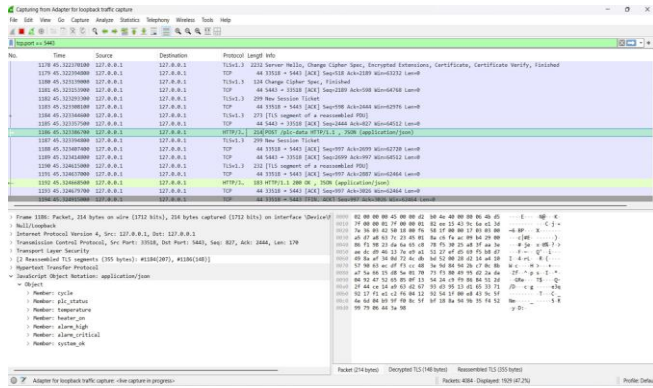


Fig 11. Wireshark capture of HTTPS traffic on port 5443, showing only TLSv1.3 encrypted data

C. TLS Decryption Verification

To verify that the data is genuinely being encrypted and not simply hidden, we use Wireshark's TLS decryption feature together with the SSL key log file generated by `plc_bridge_secure.py`. When the `sslkeys.log` file is loaded into Wireshark, it can use the recorded session keys to decrypt the captured TLS traffic and reveal the original HTTP request underneath.

As shown in Fig. 12, after loading the key log file, Wireshark labels the packet as "Decrypted TLS 148 bytes" and reveals the full JSON payload — `cycle number`, `plc_status`, `temperature`, `heater_on`, `alarm_high`, `alarm_critical`, and `system_ok` — exactly as sent by the bridge. The POST /plc-data endpoint also becomes visible again after decryption.

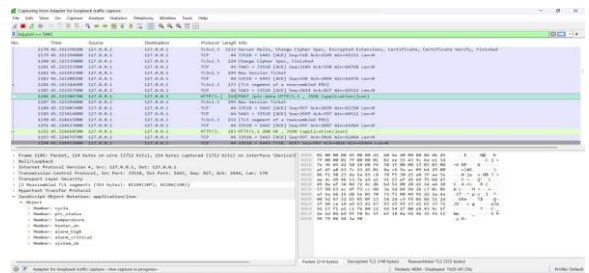


Fig. 12. Wireshark TLS decryption using the SSL key log file .

D. Summary of Results

The two scenarios produce clearly opposite outcomes. In Scenario 1, the HTTP pipeline exposes everything the protocol is plain HTTP/1.1, the full JSON payload is visible in Wireshark, the exact temperature readings and alarm states can be read by anyone on the network, and there is no authentication, no confidentiality, and no integrity protection whatsoever. Intercepting this data requires zero effort or technical skill.

In Scenario 2, the picture is completely different. The protocol is TLSv1.3, the port is 5443, and Wireshark shows nothing but encrypted binary data. No temperature values, no alarm states, no endpoint names nothing useful to an attacker. The X.509 certificate provides authentication, AES-256-GCM provides confidentiality, and the MAC ensures integrity. Without the private key, the captured traffic is completely useless to anyone who intercepts it.

The difference between the two scenarios is two lines: adding `ssl_context` to the Flask server startup, and adding `verify=CERT_FILE` to the requests call in the bridge script. This shows that strong cryptographic protection is not only effective it is also practical and easy to implement in real industrial systems.

	Scenario 1 (HTTP)	Scenario 2 (HTTPS/TLS)
Protocol	HTTP/1.1	TLSv1.3
Port	5000	5443
Data visible in Wireshark	Yes — full JSON	No — binary only
Temperature readable	Yes	No
Alarm states readable	Yes	No
Authentication	None	X.509 Certificate
Confidentiality	None	AES-256-GCM
Integrity	None	MAC Verified
Effort to intercept	Zero	Impossible without key

TABLE I: COMPARISON OF INSECURE HTTP AND SECURE HTTPS/TLS SCENARIOS

VII. RECOMMENDATIONS

Based on the results of this study, the following recommendations are made for organisations operating industrial data pipelines that connect PLCs or other OT devices to AI analytics platforms.

The most important step any organisation can take is to make TLS 1.3 mandatory for all industrial data communications. As demonstrated in this paper, unencrypted HTTP exposes the entire data pipeline to passive eavesdropping with zero technical effort. Switching to HTTPS with TLS 1.3 completely eliminates this risk, and as shown in the implementation, the change requires minimal code modification — simply adding an `ssl_context` to the server and enabling certificate verification on the client side.

Organisations should deploy a proper PKI infrastructure using CA-signed certificates rather than self-signed ones. While self-signed certificates provide the same level of cryptographic protection, they do not provide the same trust guarantees any party could generate a self-signed certificate claiming any identity. In a production environment, certificates should be issued by a trusted internal or commercial Certificate Authority, so that all devices in the network can automatically verify each other's identities without manual configuration [9].

Mutual TLS (mTLS) should be implemented wherever possible. In the scenario demonstrated in this paper, only the server presents a certificate the client (bridge script) authenticates the server, but the server does not authenticate the client. In a real deployment, both sides should present certificates, so that the Flask server can verify that the data it receives is genuinely coming from an authorised PLC bridge and not from an attacker injecting fake data into the pipeline.

Certificate lifecycle management is a critical but often overlooked aspect of PKI security. Certificates have an expiry date, and an expired certificate can cause outages or force

teams to bypass verification checks as a quick fix which defeats the purpose of having PKI in the first place. Organisations should automate certificate renewal and monitor certificate expiry dates with sufficient lead time to renew without disruption. As Dahlmanns et al. [5] showed, even devices that support TLS often leave it misconfigured or unmanaged a problem that formal certificate management processes can directly address.

Even with encrypted communications, organisations should deploy network monitoring tools such as Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) to detect anomalous traffic patterns. Encryption protects the content of communications, but it does not hide the fact that communication is happening unusual volumes of traffic, unexpected connection attempts, or connections from unknown IP addresses can all be detected and investigated even when the payload is encrypted.

Finally, all of the technical controls described above should be supported by a formal security policy aligned with ISO 27001 and ISO 27002. A security policy documents the organisation's security requirements, assigns responsibility for maintaining them, and provides a framework for auditing compliance. ISO 27002 control 8.24 specifically addresses the use of cryptography, recommending that organisations define rules for the effective use of cryptographic controls including key management, certificate management, and the selection of appropriate algorithms [10].

VIII. CONCLUSION

This paper set out to answer a straightforward question: what happens to industrial PLC data when it is transmitted without encryption, and how effectively can PKI and TLS 1.3 fix the problem. The results are clear.

In Scenario 1, transmitting PLC sensor data over plain HTTP created a complete security failure. Every packet captured by Wireshark revealed the full JSON payload in readable text temperature values, heater state, alarm flags, and system status with zero effort required from an attacker. This is not a theoretical vulnerability. As shown by multiple studies in the literature review, the majority of deployed industrial systems still communicate without encryption, leaving critical infrastructure exposed to exactly this kind of passive interception.

In Scenario 2, adding TLS 1.3 with a self-signed X.509 certificate eliminated the vulnerability entirely. Wireshark captured the same network traffic, but this time showed only encrypted binary data with no readable content whatsoever. The TLS decryption experiment confirmed that the data was genuinely encrypted and could only be recovered using the session keys logged on the server keys that in a real deployment would never be accessible to an attacker.

What makes this result particularly valuable from a practical standpoint is how little effort the transition required. The core change between the insecure and secure scenarios was two lines of code: adding `ssl_context` to the Flask server and `verify=CERT_FILE` to the bridge script. This demonstrates that the barrier to securing industrial data pipelines is not technical complexity — it is awareness and prioritisation. The tools and protocols needed to protect OT/IT communications are mature, well-documented, and easy to implement.

This study also highlights the importance of treating industrial cybersecurity as a complete system rather than a collection of isolated technical fixes. PKI provides the trust foundation, TLS provides the encrypted channel, certificate management ensures the system stays secure over time, mutual TLS extends authentication to both sides of the connection, and a formal security policy aligned with ISO 27001 and ISO 27002 ensures that all of these controls are maintained consistently across the organisation.

The findings of this paper apply directly to any organisation that transmits data from PLCs, sensors, or other OT devices to IT systems or cloud platforms which, given the pace of OT/IT convergence, now includes the majority of industrial operators worldwide. PKI and TLS are not optional extras for industrial systems. They are essential, proven, and readily available foundations for securing mission critical infrastructure.

REFERENCES

- [1] W. Alsabbagh, P. Landsmann, and M. Ploog, "Investigating the Security of OpenPLC: Vulnerabilities, Attacks, and Mitigation Solutions," *IEEE Access*, vol. 12, 2024.
- [2] R. Ma, Q. Wei, J. Wang, S. Zhu, S. Ji, P. Cheng, Y. Jia, and Q. Wang, "Towards Comprehensively Understanding the Run-time Security of Programmable Logic Controllers: A 3-year Empirical Study," *arXiv preprint arXiv:2212.14296*, 2022.
- [3] G. Barbieri, M. Conti, N. O. Tippenhauer, and F. Turrin, "Assessing the Use of Insecure ICS Protocols via IXP Network Traffic Analysis," *arXiv preprint arXiv:2007.01114*, 2020.
- [4] M. Nawrocki, T. C. Schmidt, and M. Wählisch, "Uncovering Vulnerable Industrial Control Systems from the Internet Core," *arXiv preprint arXiv:1901.04411*, 2019.
- [5] M. Dahlmanns, J. Lohmöller, J. Pennekamp, J. Bodenhausen, K. Wehrle, and M. Henze, "Missed Opportunities: Measuring the Untapped TLS Support in the Industrial Internet of Things," in *Proc. ACM Asia Conf. Computer and Communications Security (ASIA CCS)*, Nagasaki, Japan, 2022.
- [6] Z. Karimi et al., "Encryption for Industrial Control Systems: A Survey of Application-Level and Network-Level Approaches in Smart Grids," *MDPI Electronics*, vol. 6, no. 1, 2026.
- [7] C. Liu et al., "TTAS: Trusted Token Authentication Service of Securing SCADA Network in Energy Management System for Industrial Internet of Things," *Sensors*, 2021.
- [8] T. AlSaid et al., "On Design-time Security in IEC 61499 Systems: Conceptualisation, Implementation, and Feasibility," *arXiv preprint arXiv:2104.09020*, 2021.
- [9] Keyfactor, "PKI and Certificates for Industrial IoT," 2024. [Online]. Available: <https://www.keyfactor.com/blog/pki-and-certificates-for-industrial-iot-what-they-are-and-why-you-need-them/>
- [10] ISO/IEC 27002:2022, "Information Security Controls," International Organisation for Standardisation, Geneva, Switzerland, 2022.
- [11] The PowerPoint presentation accompanying this paper was created using a template designed with the assistance of Claude AI